Storage cho DevOps: block, file, object và backup

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Lưu trữ: block, file và object

Block storage trình bày các khối để hệ điều hành tạo filesystem hoặc database sử dụng; SAN là một kiến trúc kết nối storage ở mức block. File storage trình bày cây thư mục qua giao thức như NFS; nhiều client có thể mount theo quyền/cấu hình. Object storage lưu object trong bucket/namespace qua API, thường phù hợp artifact, media và backup. MinIO cung cấp object storage tương thích S3; Ceph có thể cung cấp block, file và object qua các thành phần khác nhau. Tên sản phẩm không thay thế việc chọn đúng mô hình truy cập.

Latency, throughput, IOPS, độ bền, khả năng chia sẻ, snapshot, replication và backup là các chiều đánh giá. Database nhạy với độ trễ I/O và tính nhất quán; file NFS gắn vào Pod cần hiểu access mode và failure mode; object storage không phải ổ đĩa POSIX mặc định. Một ứng dụng có thể dùng block cho database, object cho ảnh/backup và file cho chia sẻ cấu hình, nhưng cần xét chi phí và mức độ phức tạp.

Tình huống điều tra

Nhiều VM cùng chậm lúc 09:00. CPU từng VM không cao. Nếu các VM dùng chung datastore, vấn đề I/O storage là một giả thuyết đáng kiểm tra. Thu thập latency/queue của storage, thay đổi/backup job gần thời điểm đó và hiệu năng network storage. Đừng tăng CPU tất cả VM khi bằng chứng không chỉ CPU. Ngược lại, nếu chỉ một app chậm và storage chung bình thường, xem truy vấn DB, lock hoặc cấu hình VM đó.

Lab tối giản

Trên máy lab, tạo hai thư mục: một thư mục lưu file ứng dụng, một thư mục lưu bản backup. Dùng ứng dụng Notes API để ghi dữ liệu, copy bằng quy trình backup an toàn sang nơi khác, rồi restore sang file mới và xác minh. Bài này dạy nguyên tắc tách nơi lưu và kiểm tra restore; nó **không** mô phỏng được tính chất của SAN/Ceph. Nếu có môi trường VM hợp pháp, bạn có thể thêm một NFS share thử và đo tác động khi share ngắt, nhưng chỉ trong mạng lab.

Storage và ứng dụng có trạng thái

Pod có thể bị thay bất kỳ lúc nào, vì vậy file ghi trong filesystem tạm của container không phải nơi bền cho dữ liệu cần giữ. Volume cung cấp nơi lưu bên ngoài vòng đời container; PersistentVolume/PersistentVolumeClaim là mô hình Kubernetes cho storage bền theo khả năng của cluster. Nhưng "có PVC" chưa đủ: cần biết dữ liệu gắn với zone nào, có thể attach vào nhiều node không, backup/restore ra sao và điều gì xảy ra khi xóa claim. StorageClass và reclaim policy có thể quyết định volume thật còn hay mất; kiểm tra trước khi xóa.

Ứng dụng Notes API dùng SQLite, vốn phù hợp lab một tiến trình và một file. Tăng Deployment lên ba replica cùng dùng chung file SQLite qua storage tùy ý có thể tạo vấn đề đồng thời hoặc truy cập storage; đừng làm vậy chỉ để có ba Pod màu xanh. Khi hệ thống cần nhiều replica và dữ liệu chung, một database service phù hợp là lựa chọn rõ hơn. Đây là bài học quan trọng: Kubernetes không tự biến

một ứng dụng có trạng thái thành hệ thống phân tán an toàn.

Bài tập thiết kế

Vẽ ba Pod API và một database. Ghi phần nào stateless, phần nào có state. Nếu một API Pod chết, request mới có thể đi sang Pod khác qua Service, nhưng data vẫn ở database. Nếu database chết, ba Pod API vẫn có thể Running nhưng chức năng tạo ghi chú lỗi. Health/readiness nên thể hiện khả năng phục vụ theo ý nghĩa của endpoint; quyết định có làm readiness phụ thuộc DB hay không cần cân nhắc vì dependency chập chờn có thể khiến toàn bộ Pod bị loại khỏi traffic.

Backup và recovery nhiều lớp

Backup có thể là snapshot vật lý, logical dump hoặc continuous backup kèm log. Cần biết thời điểm nhất quán, retention, mã hóa/quyền, failure domain và khả năng restore sang phiên bản engine nào. Backup cùng đĩa với DB không chống mất đĩa; replica có thể nhân bản thao tác xóa sai. Nếu DB khôi phục về thời điểm cũ nhưng consumer offset ở thời điểm mới, dữ liệu có thể lệch; runbook phải chỉ cách đồng bộ hoặc replay có kiểm soát.

Diễn tập với Notes API: đặt RPO 15 phút, RTO 2 giờ; backup A, thêm ghi chú B, mô phỏng DB hỏng. Restore **vào file/instance mới**, xác nhận dữ liệu, đổi app lab sang DB mới, chạy smoke test. Ghi giờ bắt đầu-kết thúc, dữ liệu mất và điểm làm vượt RTO. Sau đó đề xuất cải tiến cụ thể về tần suất, tự động hóa hoặc vị trí bản sao.

Chọn mô hình truy cập trước chọn sản phẩm

Block storage cho hệ điều hành/filesystem/DB quản lý block; file storage qua NFS cho nhiều client đọc/ghi theo semantics file; object storage qua API lưu object/bucket. SAN mô tả hạ tầng block; Ceph có thể cung cấp block, file, object theo service; MinIO là object storage tương thích S3. Không nên mount object như POSIX rồi giả định mọi thao tác rename/lock giống filesystem. Chọn theo pattern đọc/ghi, latency/IOPS, tính nhất quán, sharing, quyền, backup, vận hành và chi phí.

Bài tập quyết định

Với Notes API: database cần ghi transaction nhỏ, có thể dùng block/PVC phù hợp; file đính kèm lớn dùng object; tập tin cấu hình chia sẻ nếu thực sự cần có thể dùng file. Nếu chỉ vì "có NFS" mà đặt database lên NFS, cần kiểm tra support/locking/durability/performance của DB và storage cụ thể. Nếu dùng Ceph, phải hiểu cluster storage có failure domain/quorum/capacity riêng; ba bản sao trên cùng một host không chống host hỏng.

PersistentVolume và trách nhiệm dữ liệu

Trong Kubernetes, PVC yêu cầu storage theo access mode/capacity/storage class; PV là tài nguyên được cấp. StatefulSet giúp định danh Pod/volume ổn định nhưng không tự backup dữ liệu. Reclaim policy có thể giữ hoặc xóa volume khi PVC bị xóa; cần đọc trước khi dọn lab. Khi Pod chuyển node, storage phải hỗ trợ attach/mount ở node mới và dữ liệu còn nhất quán. **ReadWriteOnce** không nên diễn giải thành "chỉ một Pod" trong mọi trường hợp; đọc semantics chính thức theo driver/phiên bản. Một rollout app thành công không chứng minh volume phục hồi sau mất node.

Backup khác snapshot, replication

Snapshot có thể giúp quay nhanh về điểm trước nhưng phụ thuộc hệ storage và tính nhất quán ứng dụng. Replica tăng khả dụng nhưng nhân bản lỗi logic. Backup nên tách failure domain, có retention, quyền, mã hóa và restore test. Với database, snapshot lúc transaction đang ghi có thể cần phối hợp DB hoặc log để đảm bảo phục hồi nhất quán. Diễn tập: backup A -> tạo dữ liệu B -> restore sang volume/file mới -> kiểm tra A/B theo kỳ vọng -> đo thời gian. Chỉ có file backup và chưa restore không đạt cổng năng lực.

Case: nhiều VM cùng chậm, CPU bình thường

Thu datastore latency/queue, IOPS, thời điểm backup/snapshot, network storage, log ở DB và baseline. Nếu chỉ một VM chậm, xem VM/DB riêng; nếu nhiều VM cùng datastore chậm, giả thuyết shared storage mạnh hơn. Giảm tác động có thể điều tiết job I/O lab hoặc chuyển workload theo quy trình, rồi đo lại. Không xóa snapshot/volume production để "giải phóng" khi chưa hiểu dependency và retention.

Chọn theo giao diện truy cập và tính chất dữ liệu

Block storage cho OS/DB thấy một thiết bị block để tạo filesystem; thường phù hợp DB cần I/O và semantics nhất định. **File storage** (NFS/SMB) cho nhiều client truy cập đường file theo giao thức và quyền; cần hiểu locking, cache và lỗi server/network. **Object storage** lưu object qua API/bucket/key, phù hợp backup/artifact/static asset; không là POSIX filesystem mặc định. MinIO cung cấp API tương thích S3 ở môi trường tự quản theo tính năng/phiên bản; Ceph có nhiều interface (object/block/file) với kiến trúc vận hành riêng. SAN mô tả mạng/hạ tầng block, không phải tự động tương đương backup hay HA.

Hỏi bốn câu trước khi chọn sản phẩm: app cần random write hay object PUT/GET? nhiều writer cùng file hay một writer? latency/throughput/IOPS mục tiêu? khi mất node/disk/site, dữ liệu được bảo toàn với RPO/RTO nào? SQLite trên NFS, nhiều Pod cùng ghi, có thể vấp semantics locking/performance; không “giải scale” bằng cách mount chung bừa. Một volume có thể **ReadWriteMany** về mặt storage nhưng DB vẫn chưa thiết kế multi-writer.

Từ PVC đến byte trên đĩa

Trong Kubernetes, Pod yêu cầu PVC; PVC được bind tới PV/provisioner theo StorageClass, access mode, capacity và topology. PVC **Bound** chỉ chứng minh cấp volume, chưa chứng minh dữ liệu được backup hay Pod ghi đúng. Nếu Pod Pending vì PVC, đọc Events của PVC/Pod, StorageClass, provisioner, node topology và quota. Nếu Pod chạy nhưng ứng dụng báo permission denied, xem UID/GID, filesystem ownership và securityContext; đổi access mode không chữa quyền file. Nếu Pod chuyển zone trong khi volume chỉ gắn được một zone, scheduler/topology có thể ngăn placement hoặc khiến recovery kéo dài.

Reclaim policy quyết định tài nguyên storage sau khi PVC/PV bị xóa theo cấu hình; không dùng một giá trị mặc định làm giả định bảo vệ dữ liệu. Một thao tác **helm uninstall** có thể không xóa PVC hoặc có thể kéo theo dữ liệu tùy chart/policy; trước khi dọn lab, kiểm tra resource thực. Object storage lifecycle/retention cũng có thể xóa backup ngoài ý muốn nếu cấu hình sai.

Backup, snapshot, replication và restore

Snapshot ghi lại trạng thái ở một thời điểm theo khả năng hệ thống; nếu cùng failure domain với dữ liệu gốc, nó không bảo vệ trước mất cả storage. Replication giúp có bản sao khi phần cứng hỏng nhưng lệnh xóa hoặc dữ liệu lỗi cũng có thể được nhân bản. Backup là bản sao độc lập theo retention/immutability/khả năng restore được thử. Với DB, cần transaction-consistent backup, log/WAL/binlog nếu muốn point-in-time recovery, kiểm tra checksum và một restore drill ở môi trường cách ly. RPO/RTO là kết quả đo, không phải thông số brochure.

Ví dụ backup mỗi 6 giờ, không có log liên tục: trong trường hợp xấu mất gần 6 giờ dữ liệu. Nếu copy 2 TB qua đường mạng thực 100 MB/s, riêng truyền lý tưởng mất khoảng 5,6 giờ (2.000.000 MB / 100 MB/s), chưa tính download, decompress, index rebuild, kiểm tra app. RTO 1 giờ không phù hợp nếu không có standby/replication hoặc đường phục hồi khác. Tính toán đơn giản giúp phát hiện lời hứa phi thực tế trước khi sự cố xảy ra.

Lab mô hình trên máy riêng

Tạo một file SQLite lab và một bản sao độc lập bằng cơ chế backup phù hợp của SQLite/app, không chỉ **cp** file khi DB đang ghi. Ghi note A, backup, ghi note B, rồi restore bản backup vào **đường khác** để xác nhận bản restore có A và có/không có B theo thời điểm backup. Không ghi đè DB gốc trong bài. Ghi thời gian tạo backup, kích thước, checksum và thời gian restore. Nếu app mẫu đang dùng nhiều process, dừng ghi hoặc dùng SQLite backup API để có snapshot nhất quán; đọc tài liệu chính thức theo phiên bản. Dọn file restore riêng sau khi đã đối chiếu.

Lỗi có chủ đích trên giấy: PVC đầy, Pod vẫn **Running**, API trả 500 khi ghi. Kiểm tra dung lượng/inode ở filesystem container và storage backend, log DB, PVC capacity/expansion policy, metric storage. Giảm tác động bằng ngăn log/temp phình, mở rộng có kế hoạch hoặc khôi phục theo runbook; không xóa dữ liệu nghiệp vụ để có chỗ mà chưa biết retention. Nếu nhiều VM cùng chậm, tìm shared datastore/network thay vì tăng CPU từng VM.

Câu hỏi: ba replica Ceph có nghĩa không cần backup? Không; replication chống một số lỗi hạ tầng, không chống xóa logic/ransomware như backup độc lập. **Câu hỏi: ReadWriteMany** có cho ba MySQL instance cùng ghi một data directory? Không; access mode chỉ mô tả khả năng mount, không thay cơ chế DB cluster. Nguồn: Kubernetes storage, Ceph documentation, SQLite backup API.

Consistency: bản sao phải đại diện cho thời điểm nào?

Khi một DB đang ghi, copy trực tiếp file dữ liệu có thể lấy các trang ở những thời điểm khác nhau, tạo bản sao không nhất quán. Snapshot ở storage có thể là crash-consistent, tương đương máy mất điện tại một thời điểm; DB engine có thể recovery bằng log nếu snapshot gồm đúng tập volume/log và quy trình phù hợp. Application-consistent backup thường phối hợp DB flush/backup API/transaction checkpoint để khi restore dữ liệu có nghĩa. Trước khi chọn “snapshot nhanh”, hỏi workload có nhiều volume không, log/WAL ở đâu, có cần quiesce không, sau restore đã chạy kiểm tra tính nhất quán chưa. Đây là lý do backup policy phải do đội app/DB và storage cùng thiết kế.

Với object storage, versioning/immutability giúp chống xóa/ghi đè một số tình huống nhưng cũng tăng chi phí/retention; IAM sai vẫn có thể khiến dữ liệu lộ. Multipart upload đang dở có thể tốn chỗ; lifecycle

cần kiểm tra. Với NFS, network latency/locking và server HA ảnh hưởng app; “có file share” không nghĩa mọi workload được scale nhiều writer. Với SAN, multipath và zoning có thể giúp đường I/O dự phòng, nhưng phải thử mất một path và đo failover. Không nên giới thiệu một giao thức là nhanh/bền tuyệt đối: hiệu năng phụ thuộc workload, cấu hình và phần cứng.

Bài tập capacity và RTO

Giả sử DB 1,5 TB, tăng 100 GB/tháng, backup full hằng tuần giữ 4 tuần và incremental hằng ngày 6 ngày/tuần. Nếu full không nén, riêng 4 bản full là 6 TB, chưa tính incremental, metadata, snapshot, bản sao offsite và growth. Tính lại với tỷ lệ nén thực đo được, không lấy con số quảng cáo. Nếu restore throughput hữu dụng chỉ 150 MB/s, truyền 1,5 TB lý tưởng đã gần 2,8 giờ, chưa tính replay log và kiểm tra app. RTO 1 giờ đòi standby/replication hoặc phương án khác; một backup full mỗi tuần không thể tự đáp ứng.

Bài tập lựa chọn: log ứng dụng giữ 30 ngày, ảnh người dùng, data directory PostgreSQL và backup DB nên đặt vào block/file/object loại nào? Gợi ý: log có thể đưa collector tới object/index backend; ảnh hợp object API; PG data thường cần block/filesystem được engine hỗ trợ với latency/consistency phù hợp; backup phù hợp object/offsite theo công cụ. Không có đáp án duy nhất nếu thiếu RPO/RTO, throughput, quy mô và quyền; bạn phải nêu giả định và test.

Ceph, MinIO, NFS: học trách nhiệm vận hành

Ceph phân phối dữ liệu qua cluster và có interface block/file/object, nhưng yêu cầu thiết kế failure domain, network, capacity headroom, recovery traffic, monitoring và upgrade. Một cluster có 3 node nhưng cùng rack/power chưa chịu lỗi rack. Khi một disk chết, recovery/replication có thể làm latency tăng; cần capacity và bandwidth để phục hồi mà không làm app quá tải. MinIO object storage tự quản cần xem erasure coding, drive/node failure, bảo vệ bucket, lifecycle, versioning, backup/DR theo phiên bản triển khai. NFS cần server/storage HA và kiểm tra mount semantics. Đừng nói “cài được” thành “vận hành được”.

Runbook cho PVC Pending

- Xác định PVC nào, namespace, StorageClass và event **ProvisioningFailed**/**WaitForFirstConsumer**/quota. 2. Đọc CSI controller/node plugin logs theo resource cụ thể, kiểm tra topology và quyền backend. 3. Nếu PVC Bound nhưng Pod Pending, xem node affinity/attach/mount và lỗi volume multi-attach. 4. Chỉ thay StorageClass/capacity sau khi hiểu dữ liệu đang ở đâu và thao tác có tạo volume mới hay không. 5. Sau sửa, kiểm tra Pod mount, app đọc/ghi, dữ liệu cũ còn, metric latency và backup. Không xóa PVC production để “tạo lại” nếu chưa có restore plan.

Câu hỏi: Vì sao **kubectl get pvc** Bound chưa đủ? Nó chỉ chứng minh binding; mount, quyền, consistency, hiệu năng và backup chưa được kiểm tra. **Vì sao replication không chống lỗi người dùng xóa bảng?** Lệnh xóa logic cũng được nhân bản. **RPO/RTO là lời hứa cho ứng dụng hay storage?** Cho dịch vụ/dữ liệu người dùng; storage chỉ cung cấp một số cơ chế để đạt mục tiêu. Nguồn: Kubernetes persistent volumes, Ceph architecture.

Bài tập phục hồi: chọn một backup, ghi checksum và thời điểm, restore sang namespace/VM cách ly, chạy query kiểm tra ba bản ghi đã biết và một request app. Nếu chỉ mount được volume nhưng schema sai hoặc dữ liệu thiếu, restore chưa đạt. Đo thời gian từng bước để so với RTO, ghi thiếu sót và sửa runbook trước khi công bố mục tiêu phục hồi.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://docs.ceph.com/en/latest/
- https://kubernetes.io/docs/concepts/storage/
- https://min.io/docs/minio/linux/operations/concepts.html